Beyond Sanitizers: LLM-Guided Refinement for Differential JSON Deserialization Testing in Dart

Ziyad Mohammad Mansy Ibrahim
Independent Researcher
ziyadmohammad37@gmail.com
ORCID: 0009-0008-3499-3828

Abstract—A prior black-box, LLM-guided grammar-fuzzing pipeline raised a native C parser's acceptance rate from 58.2% to 97.1% using only coverage-free feedback, and named a memory-safe target – where "crash" has no sanitizer-based meaning – as future work along a materially different generalization axis than swapping one C parser for another. We take up that challenge directly. We retarget the identical refinement loop at four widely-used Dart/Flutter JSON deserialization paths – hand-written parsing, and the json_serializable, freezed, and built_value codegen frameworks – and replace the sanitizer-based crash oracle with two oracles that need no crash at all: cross-implementation differential divergence, and a ground-truth numeric round-trip check. Across a budget-scoped n=5-per-arm design, refinement reproduces the original result almost exactly on a like-for-like acceptance objective (50.0% to a 92.0% mean, complete rank separation, exact two-sided $p \approx 0.0079$), confirming the underlying mechanism transfers to a new runtime and language. The same mechanism does not transfer to our differential objective: even after correcting the refinement prompt to isolate divergence as the explicit score – itself a confirmed, three-seed fix, not a guess – refinement finds cross-implementation divergence in a 2.9% mean of generated documents, against 22.2% for a simple static, schema-aware generator (complete separation, exact two-sided $p \approx 0.0079$, opposite direction from the acceptance result). We test, rather than speculate about, the obvious alternative explanation – that the gap is generation overhead from a sandbox that forbids the LLM from importing a JSON library, forcing it to hand-build JSON text – by re-running the refined arm with that one restriction lifted: the resulting arm reaches 100% syntactically valid output, essentially eliminating the overhead, yet its divergence rate (2.2% mean) is statistically indistinguishable from the constrained sandbox's ($p \approx 0.69$) and remains just as far below the baseline. The RQ2 shortfall is a targeting problem, not a generation-quality one, decisively – and richer feedback does not fix it either: telling the proposer which categories of perturbation its divergent documents exhibited, rather than a single score, performs measurably worse (0.95% mean, $p \approx 0.032$ against the score-only prompt), a genuine data point against the assumption that more informative feedback is strictly better for this task. The static generator's own campaigns surface two concrete, practitioner-actionable defects independent of any LLM: built_value silently defaults a missing list-typed required field to empty rather than rejecting the input, while json_serializable and freezed silently saturate out-of-range integer fields to int64::MIN/MAX with no exception at all – confirmed at a 5.7% mismatch rate across 43,336 sampled records via a no-cost analysis pass over already-collected campaign data, rather than a fourth, redundant fuzzing pipeline. We argue the resulting picture – refinement's benefit is objective-dependent, not universal, even holding target, model, and iteration budget fixed – is itself the more useful methodological contribution than either number in isolation, and release the full pipeline, harness, and campaign data.

Index Terms—fuzzing, differential testing, property-based testing, large language models, JSON deserialization, Dart, Flutter, black-box testing

I. Introduction

A black-box, LLM-guided grammar-fuzzing pipeline can be evaluated along two different generalization axes. One is swapping the target implementation while keeping everything else fixed – a prior study did this, reusing an identical pipeline unmodified against a second, independently implemented C JSON parser, and found the mechanics of the loop (harness contract, proxy signal, sandbox) transferred with no redesign [1]. The other axis, which that same study named explicitly as unaddressed future work, is swapping the oracle itself: every component of that pipeline's failure signal – a sanitizer-instrumented build, a subprocess's exit code, a fatal signal – assumes a native process that can crash in the memory-unsafe sense. A memory-safe language has no such crash to look for, and "failure" has to be redefined around something else entirely: uncaught exceptions, silently violated API contracts, or values that decode incorrectly without any exception at all.

This paper takes up that second axis directly, against a target chosen for practical relevance rather than novelty for its own sake: JSON deserialization in Dart, the language Flutter mobile and web applications are written in. A first attempt at porting the original pipeline's crash oracle onto a single, well-audited Dart JSON decoder would likely reproduce the same kind of result the original study already reported for its second C target – a negative result, explainable but not informative, since a heavily-used standard-library decoder is exactly the kind of target unlikely to crash under a bounded fuzzing budget. We reject that port and design two oracles instead that do not depend on a crash occurring at all. The first is differential divergence: rather than one implementation whose accept/reject decision is the only signal available, we run every generated document through four independently-implemented, widely-used Dart JSON deserialization paths simultaneously – hand-written parsing,

and the json_serializable, freezed, and built_value codegen frameworks – and treat disagreement between them, not a crash, as the signal. The second is a ground-truth numeric round-trip check: JSON permits arbitrary-precision integer literals syntactically, and we check whether a decoded value preserves the exact input magnitude, which needs no assumption about what "correct" looks like beyond the literal text the generator wrote.

Retargeting the oracle this way changes what the refinement loop is actually being asked to optimize, and we treat that as an empirical question rather than assuming the original result transfers. Concretely, this paper asks three research questions. RQ1: reusing the original acceptance-rate proxy and refinement loop essentially unmodified, does LLM-guided refinement still raise a Dart JSON generator's rate of producing syntactically valid documents, the way it did for the original C target? RQ2: across the four Dart deserialization paths, does malformed or boundary-case input surface behavioral divergence, and does LLM-guided refinement – with its prompt corrected to make divergence the explicit objective, not just an incidental metric shown alongside others – find more of it than a static, schema-aware generator designed by hand after studying a small number of cases? RQ3: does numeric decoding across these four paths preserve exact integer values at magnitudes JSON's grammar permits but native integer types do not, and how far does this generalize once measured at scale rather than on one hand-picked probe?

We answer RQ1 affirmatively and RQ2 negatively, and the contrast between the two is the paper's central finding: the same refinement mechanism, same proposer model, same iteration budget, transfers cleanly to one objective and not to the other. RQ3's answer is a concrete, quantified, and – to our knowledge – previously undocumented defect pattern across two of the four most common Dart serialization frameworks in real use. Concretely, this paper contributes:

- Two coverage-free, crash-free oracles – cross-implementation differential divergence and ground-truth numeric round-trip checking – designed specifically for a memory-safe target where the original pipeline's sanitizer-based crash oracle does not apply, and a two-stage harness contract (a shared JSON-syntax gate, then four independent schema-level attempts) that makes the divergence oracle computable automatically from four otherwise-ordinary Dart programs.
- A budget-scoped, seeded, n=5-per-arm empirical comparison (RQ1) showing the original pipeline's refinement mechanism reproduces its acceptance-rate effect almost exactly on a new runtime and language – 50.0% to a 92.0% mean, complete rank separation, exact two-sided $p \approx 0.0079$ – a direct generalization result along the axis the original study left open.
- A second n=5-per-arm comparison (RQ2) showing the same mechanism, applied to a differential divergence objective instead, underperforms a static, schema-aware generator by an order of magnitude (2.9% mean vs. 22.2% mean, complete rank separation in the opposite direction, exact two-sided $p \approx 0.0079$).
- A follow-up ablation that tests, rather than assumes, why: lifting the sandbox's restriction against importing a JSON library eliminates the LLM's generation-quality overhead almost entirely (every run reaches at or near 100% syntactically valid output) yet leaves the divergence rate statistically unchanged (2.2% mean, $p \approx 0.69$ against the constrained sandbox) – establishing decisively that RQ2's shortfall is a targeting problem, not a generation-quality one.
- A second follow-up testing the natural next hypothesis – that richer feedback (perturbation categories, not just a score) closes the targeting gap – and finding the opposite: a measurably worse divergence rate (0.95% mean, $p \approx 0.032$ against the score-only prompt), a genuine negative data point against the assumption that more informative feedback is strictly better.
- A methodological note, confirmed rather than assumed: an initial, unmodified refinement prompt found zero divergence across three full seeded runs despite driving up unrelated rejection counts; a principled prompt revision that isolates divergence as an explicit score, validated on a held-out seed before any further spend, recovered a nonzero and consistent effect. Reporting the failed first design alongside the fix is itself part of this contribution.
- Two concrete, practitioner-actionable defects (RQ3 and part of RQ2), found by a static generator with no LLM involved and confirmed at scale by re-analyzing already-collected campaign data rather than running a redundant pipeline: built_value silently defaults a missing list-typed required field instead of rejecting the input, and json_serializable/freezed silently saturate an out-of-range integer field to int64::MIN/MAX with no exception, at a measured 5.7% mismatch rate across 43,336 sampled checks spanning every nesting depth the schema's recursion produced.
- A released pipeline, harness, and full campaign artifact set, reusing the original project's sandboxing, seeding, and reproducibility-manifest machinery entirely unmodified where it is not C- or format-specific, and documenting precisely where it needed to change and where it did not.

The rest of the paper is organized as follows. Section II situates this work relative to coverage-guided, grammar-based, differential, and LLM-assisted testing. Section III describes the oracle design, the four-path harness, the generators, and what is reused unmodified from the original pipeline. Section IV reports all three research questions. Section V discusses the RQ1/RQ2 contrast, threats to validity, and future work. Section VI concludes.

II. Related Work

a) Coverage-guided fuzzing.: American Fuzzy Lop [2] and its successor AFL++ [3] popularized mutation-based, coverage-guided fuzzing: a target is instrumented to report which code ran, and the fuzzer retains and mutates inputs that reach new coverage. Manès et al.'s survey [4] gives a taxonomy of this and related approaches. Coverage-guided fuzzers are highly effective when instrumentation is available, but instrumentation itself is exactly what a memory-safe, JIT/AOT-compiled runtime such as Dart's makes both less necessary (there is no memory-safety class of bug for a sanitizer to catch) and less available in the form these tools assume; none of this family targets the failure modes this paper is concerned with.

b) Grammar-based and generation-based fuzzing.: Generating inputs directly from a grammar, rather than mutating bytes, is a much older idea: Purdom's sentence generator [5] produces short strings that exercise every grammar production, originally for testing and debugging parsers themselves. More recent tools combine a grammar with coverage feedback: Superion [6] extends a greybox fuzzer with grammar-aware mutation operators so mutated inputs remain syntactically valid, and Nautilus [7] similarly pairs a grammar-based generator with coverage-guided mutation to reach deeper parser states. Both still assume coverage instrumentation is available; the pipeline this paper extends removes that assumption entirely.

c) Property-based testing.: QuickCheck [8] introduced property-based testing: a developer writes generators for values of a given shape and a property that should hold for all of them, and the tool searches for a counterexample. Hypothesis [9], which the pipeline this paper reuses builds directly on, is a Python implementation of the same idea, with composable strategies expressive enough to encode both a formal grammar (our RQ1 generator) and an adversarial perturbation catalog over a target-specific schema (our RQ2 generator, Section III).

d) LLMs for fuzzing and test generation.: LLMs have recently been used to generate or mutate fuzzing inputs directly. TitanFuzz [10] uses an LLM to generate and mutate programs for fuzzing deep-learning library APIs, relying on the LLM's prior exposure to real usage examples of those APIs rather than a formal grammar. Fuzz4All [11] generalizes this to arbitrary languages/systems by using an LLM both to synthesize a distilled input prompt from documentation and examples and to generate and mutate inputs from it. CodaMOSA [12] invokes an LLM only after search-based test generation's coverage plateaus, asking it for example test cases to escape the plateau – i.e., the LLM is invoked because coverage feedback is available and has stalled. This paper's setting differs along the same axis as its predecessor [1]: no coverage signal is available at any point, and the LLM's job is to revise a reusable, executable generator against feedback a black-box harness can actually produce, not to emit one-off inputs.

e) Differential testing.: Running multiple independent implementations of the same specification against identical inputs and comparing their outputs, rather than checking each against a hand-written oracle, dates to McKeeman's differential testing [13]. This idea is what makes RQ2's oracle possible at all in a setting with no crash to detect: four independently-implemented Dart JSON deserialization paths sharing one formal schema give a comparison target for free, the same way McKeeman compared independent compiler implementations against each other rather than against a formal semantics. Where classic differential testing compares whole programs given the same input, our four paths share almost their entire input pipeline (Section III's shared JSON-syntax gate) and differ only in schema-level decoding, which is a narrower and, we find, more failure-prone surface than differential testing is usually applied to.

f) Positioning.: This paper is a direct empirical follow-up to [1], which evaluated the identical refinement-loop mechanism on two independently implemented C JSON parsers under a sanitizer-based crash oracle and named a memory-safe target as future work along a materially different generalization axis. We take up exactly that axis: same loop, same sandboxing discipline, same reproducibility tooling where none of it is C- or format-specific, retargeted at a language where the entire premise of a crash oracle does not hold, using differential divergence [13] and ground-truth numeric round-tripping as replacements. Unlike TitanFuzz, Fuzz4All, and CodaMOSA, the LLM here never sees coverage data, execution traces, or usage examples – only a schema description and the same three-signal proxy discipline (here, a divergence score in place of acceptance rate) a human maintaining a Hypothesis strategy by hand would have to reason about manually.

III. Methodology

A. Research questions

RQ1: reusing the original acceptance-rate proxy and refinement loop unmodified in mechanism, does LLM-guided refinement raise the fraction of generated documents a Dart JSON decoder accepts as syntactically valid, relative to a static grammar-only baseline, the way it did for a native C target? RQ2: across four independently-implemented Dart JSON deserialization paths, does malformed or boundary-case input surface behavioral divergence, and does refinement – once its prompt is corrected to make divergence the explicit objective – find more of it than a static, schema-aware generator? RQ3: does numeric decoding across these paths preserve exact integer values at magnitudes JSON permits syntactically but a 64-bit integer type does not, and how does this generalize once measured at scale?

B. Target set

Four Dart [14] JSON deserialization paths over one shared schema (Section III-C), chosen for being how a real Flutter codebase actually deserializes JSON today, not for novelty: (A) hand-written parsing via dart:convert's jsonDecode plus explicit as casts, representing code with no generated safety net; (B) json_serializable 6.14.1 [15], the most widely used Dart codegen serialization package; (C) freezed 4.0.1 [16], configured to delegate JSON leaf-decoding to json_serializable so that any B-vs-C divergence is attributable to freezed's own contribution (immutability, union discrimination) rather than a second independent JSON decoder; (D) built_value 8.13.0 [17], an independently-designed, typed-serializer-based model rather than an annotation-and-Map-based one like B/C, and consequently the path most likely to diverge for genuinely independent-implementation reasons. All four resolve together under one Dart 3.13.2 toolchain with no dependency conflicts; exact versions are pinned in a committed lockfile.

C. Shared schema

One logical record, hand-implemented once per path: id (a bare JSON integer, RQ3's target field), amount (a JSON string, carrying text byte-for-byte with no numeric coercion by any path, so it cannot exhibit RQ3's failure mode – it exists purely to widen RQ2's perturbation surface), name (nullable string), status (one of three enumerated string values, chosen to probe unrecognized-enum-value handling), tags (a list of strings), and child (one bounded level of recursive nesting of the same record).

D. Two coverage-free, crash-free oracles

Every generated document passes through a two-stage harness. Stage one, a shared structural gate: base64-decode the candidate bytes, decode as UTF-8, then attempt jsonDecode. Any failure here – invalid UTF-8, invalid JSON syntax, or a syntactically valid but non-object top-level value (a bare array, number, or null, which JSON's grammar permits but no path can schema-decode) – is recorded once, uniformly, since all four paths share this exact entry point and would fail identically before ever diverging. This has a direct consequence for generator design (Section III-G): a byte-level, grammar-only mutator can vary only what this shared gate tests, so it cannot find divergence by construction – divergence can only exist at the schema stage. Stage two, four independent schema attempts, reached only if stage one succeeds: each of the four paths attempts to decode the same parsed object, independently caught so that one path's exception can never prevent the other three from running on the same input. Each result is either accepted, with a canonical value (Section III-E), or rejected, with the concrete runtime type of the exception and whether that type is private – Dart's leading-underscore naming convention marking a class as internal to its defining library, meaning calling code cannot name it in a catch clause and must catch a generic supertype instead. This operationalizes "undocumented failure surface" mechanically rather than via a hand-maintained list of each package's documented exceptions.

From these four records, two divergence-based oracles are computed automatically, with no crash required: cross-path divergence (RQ2) is accept/reject when acceptance status differs across paths, or value when two or more accepting paths decode to different canonical values. Numeric round-trip divergence (RQ3) compares each accepting path's canonical id against the exact ground-truth integer parsed from the candidate's own source text – Python's arbitrary-precision int serializes exactly via json.dumps with no float detour, so ground truth requires no special-cased generation, only ordinary integer drawing at a biased range. A genuine process-level fault (the harness killed by a signal, or a batch timeout) is this pipeline's closest analogue to the original's sanitizer-based crash tier, checked around a whole campaign rather than around one input (Section III-F).

E. Canonical value comparison

Comparing decoded values safely requires that comparison itself not reintroduce the exact class of error the oracle exists to detect. Each path's typed result is converted, once, to a shared representation using only Dart primitives – strings, exact-precision integers, booleans, null, and nested maps/lists – and compared structurally, never as a floating-point value. Any reported divergence is therefore real by construction, not an artifact of the comparison logic.

F. Harness architecture

The original pipeline spawns one process per input, which is cheap for a single native binary but would be four times as expensive here and provides no fault-isolation benefit a memory-safe language cannot already give in-process (an exception is caught, not a process-ending fault, unless truly fatal). The harness is instead one AOT-compiled, long-lived process per campaign, reading newline-delimited JSON requests from stdin – one line per candidate, base64-encoded since a candidate may be invalid UTF-8 by design – and writing one newline-delimited JSON result per line, flushed immediately so a driver can still attribute a later process-level fault to the correct input despite one process now serving an entire campaign rather than one input.

G. Generators

RQ1 reuses the original pipeline's grammar-derived generator (the ANTLR grammars-v4 JSON grammar [18], [19] translated by hand into Hypothesis combinators) completely unmodified, targeting the shared structural gate directly: "accepted" is defined exactly as the original grammar defines it – any value type at the top level,

not only objects – specifically so this research question tests portability of the refinement mechanism on a like-for-like criterion, not a criterion this paper's own schema happens to impose. RQ2/RQ3 require a schema-aware generator instead, since Section III-D's shared gate makes a grammar-only mutator structurally incapable of finding schema-level divergence: a base valid-record generator draws each field at its correct type, with id biased toward magnitudes near double's exact-integer boundary and 64-bit integer boundaries; an independently-probable perturbation layer then drops a field, substitutes a wrong-but-JSON-valid type, nulls a field regardless of nullability, substitutes an out-of-enum string, injects an unknown key, or forces deeper recursion than the base generator's normal cap. For the LLM-refined arm, the identical @st.composite def generated_json(draw) -> bytes contract is used, with a schema description substituted for the grammar text in the prompt and, for RQ2, a divergence score (Section III-I) in place of acceptance rate as the quantity the prompt asks the model to maximize.

H. What transfers unmodified from the original pipeline

The original project's AST-based sandbox over LLM-authored proposals (import restricted to hypothesis.strategies, a blocklist over named code-execution/I-O entry points, and a behavioral sanity check that the proposal actually produces bytes); its bounded iteration budget and error-recovery discipline (a failing proposal consumes an iteration slot and its exact error text is fed into the next prompt, rather than aborting the run); its Hypothesis reproducibility shim, re-verified against the exact pinned Hypothesis version this shim's private internal requires (a newer version already lacked the attribute, reproducing the original's own documented fragility first-hand); and its per-iteration artifact discipline (every prompt, proposal, and result persisted, nothing discarded) are all reused with no logic changes, only the target-specific campaign runner beneath them replaced. The structural-fingerprint diversity proxy is likewise reused verbatim: it is JSON-specific, not C-specific, and the format has not changed.

I. RQ2's refinement prompt: a confirmed fix, not a guess

An initial prompt presented the divergence count alongside per-path rejection counts in one undifferentiated metrics object, with no relative weighting. Across three full seeded runs this produced zero divergence despite driving rejection counts up sharply – confirmed as systematic, not run-to-run noise, before any prompt change was made (Section IV-B). The prompt was revised to state an explicit numeric score (divergence count over total), to say plainly that a high rejection count alone is not the goal, and to add a methodological hint – vary one or two things about an otherwise-valid document rather than maximizing how broadly malformed a document looks – deliberately worded to avoid naming the specific fields or outcomes already known by hand, so that any resulting discovery would be the loop's own. This fix was validated on a single held-out seed before being adopted for the reported comparison (Section IV-B), the same discipline as validating a build before spending a full experimental budget on it.

J. Ablation: is the RQ2 gap generation overhead or targeting?

Section III-D's sandbox forbids the LLM from importing json, forcing it to hand-build JSON text, which is failure-prone. To isolate how much of RQ2's gap (Section IV-B) this overhead accounts for versus a genuine targeting shortfall, we ran a second n=5 refined arm under an otherwise-identical sandbox and prompt that additionally permits import json and states so explicitly, letting the proposer call json.dumps on an ordinary Python structure instead of concatenating strings by hand. This is implemented as a deliberately separate sandbox module rather than a parameter on the original's validator, so the original – reused unmodified for the main RQ1/RQ2 comparison – is never at risk from this one ablation.

K. Follow-up: category feedback, not just a score

Section III-J established the RQ2 gap is a targeting problem, not generation overhead, which raises a direct follow-up: would telling the proposer which categories of perturbation its divergence-causing documents exhibited help it target more effectively than the single numeric score alone? Categories – missing field, null override, wrong type, unrecognized enum, boundary-magnitude id, extra key, deep nesting – are computed automatically by inspecting each divergence-causing document's shape against the schema, independent of which generator produced it (the LLM's own generator source is not something this pipeline can introspect), and are fed back as counts without naming specific fields, preserving Section III-I's discipline against planting the answer.

L. Experimental design

All three research questions use seeded, independent runs (Hypothesis's draw stream reseeded once per run via the reproducibility shim above), with n=5 per arm rather than a larger sample: real API spend against a fixed budget makes n=5 the responsible choice here, reported honestly as such throughout rather than implied to carry n=15's statistical power – the same discipline the original project applied to its own smaller-sample feedback ablation. RQ1 compares a static baseline (one iteration, the grammar generator alone) against five refined runs (five iterations of up to 500 examples each, gpt-4.1-mini, temperature 0.2). RQ2 compares a static baseline (the schema-aware generator, one campaign of 500 examples per run) against five refined runs under the corrected prompt. RQ3 requires no separate campaign: every schema-evaluated record collected for RQ2 already

TABLE I
RQ1: acceptance rate, n=5 seeded runs per arm.

Metric	Value
Baseline, per-run (%)	50.0, 50.0, 50.0, 50.0, 50.0
Refined, per-run (%)	68.6, 97.3, 98.0, 98.1, 98.2
Baseline mean (stdev)	50.0 (0.0)
Refined mean (stdev)	92.0 (13.1)

TABLE II
RQ2: cross-path divergence rate, n=5 seeded runs per arm. Baseline uses the static schema-aware generator; refined uses the corrected prompt.

Metric	Value
Baseline, per-run (%)	19.8, 21.8, 22.2, 23.4, 23.8
Refined, per-run (%)	1.3, 1.45, 2.75, 3.6, 5.53
Baseline mean (stdev)	22.2 (1.57)
Refined mean (stdev)	2.93 (1.74)

carries each path's canonical id and the document's own source text, so RQ3's ground-truth comparison is computed as an analysis pass over RQ1/RQ2's existing data rather than a third parallel generator/harness/campaign pipeline.

IV. Evaluation

All reported p-values are exact, two-sided Mann-Whitney U tests (scipy.stats.mannwhitneyu, method="exact"), computed directly rather than via the closed-form shortcut a fixed-budget design can tempt one into approximating by hand; at n=5 per arm with complete rank separation, $C(10, 5) = 252$ and exactly two of those partitions produce complete separation in either direction, so $p = 2/252 \approx 0.00794$ is the exact value in every complete-separation case reported below, not a coincidence between unrelated effects.

A. RQ1: does the refinement mechanism transfer?

Table I reports acceptance rate – the fraction of generated documents that are syntactically valid JSON by the same criterion the original grammar uses, any value type at the top level – for five seeded baseline runs and five seeded refined runs. The baseline is exactly 50.0% on every run by construction (its input stream deterministically alternates a grammar-valid generator with a deliberately corrupted variant of it). Every refined run exceeds every baseline run: complete rank separation, exact $p \approx 0.00794$. One refined run's lower rate (68.6%) traces to a seed on which only one of five iterations produced usable data, the rest consumed by ordinary LLM code-generation failures (e.g. one proposal built an unbounded-depth strategy that hit Python's recursion limit) – logged and handled exactly as the original pipeline's error-recovery design specifies, not a defect in this port.

This reproduces the original study's cJSON result in shape – 58.2% to 97.1% there, 50.0% to a 92.0% mean here – on a completely different runtime and language, with the identical refinement-loop mechanism and no target-specific changes to the loop itself. We read this as confirmation that the mechanism, not merely the original numeric result, generalizes.

B. RQ2: does refinement find more divergence than a static generator?

The first refinement prompt (Section III-I) was run across three full seeded runs (seeds 0-2) before any revision: zero cross-path divergence in every one of the resulting iteration-campaigns, despite per-path rejection counts climbing sharply by the final iteration of each run (up to 208 of 264 schema-evaluated documents on one run). This ruled out single-run variance as an explanation before the prompt was changed. The revised prompt (Section III-I) was validated on one held-out seed (11.8% divergence in its best iteration, against a confirmed 0% baseline under the old prompt) before being adopted for the five-seed comparison reported in Table II.

Every baseline run exceeds every refined run: complete rank separation, exact $p \approx 0.00794$, in the opposite direction from RQ1. Restricting the comparison to only the subset of each run's generated documents that reached the schema stage at all (i.e. excluding the refined arm's additional overhead from syntactically invalid JSON, since the sandbox forbids importing a JSON library and the LLM must hand-build JSON text) still leaves a wide gap: a 4.10% mean divergence rate among schema-evaluated documents for the refined arm, against the baseline's 22.2% (identical to its overall rate, since the static generator always emits valid JSON). Generation-quality overhead is real – the refined arm's schema-evaluated fraction of all generated documents ranged 41.9%-83.7% across runs, versus the baseline's fixed 100% – but this comparison alone cannot separate how much of the gap it explains from a targeting shortfall; Section IV-C isolates the two directly. Pooling every schema-evaluated record across all five refined seeds, the loop found exactly the same four divergence patterns the static generator found (including both confirmed defects below), never a qualitatively new one, at lower absolute rate throughout.

C. Ablation: generation overhead, or targeting?

Table III reports the ablation from Section III-J: an otherwise-identical refined arm, five fresh seeds (100-104, disjoint from every other reported run), with import json additionally permitted. Every one of the 24 iterations that produced data reached 100% (or, in one case affected by an unrelated encoding issue, 82.5%) schema_evaluated – the generation-overhead problem is essentially eliminated, exactly as expected once the LLM no longer needs to hand-roll JSON text.

The divergence rate did not improve: 2.20% mean, statistically indistinguishable from the constrained sandbox's

TABLE III
Ablation: divergence rate with import json permitted, n=5 fresh seeded runs, vs. the main text's constrained-sandbox refined arm and static baseline.

Metric	Value
json-allowed, per-run (%)	0.05, 2.2, 2.36, 2.36, 4.05
json-allowed mean (stdev)	2.20 (1.42)
Constrained-sandbox mean (stdev)	2.93 (1.74)
Static baseline mean (stdev)	22.2 (1.57)

TABLE IV
Follow-up: divergence rate with perturbation-category feedback, n=5 fresh seeded runs, vs. the constrained-sandbox refined arm.

Metric	Value
Category-feedback, per-run (%)	0.0, 0.0, 1.0, 1.2, 2.55
Category-feedback mean (stdev)	0.95 (1.05)
Constrained-sandbox (score-only) mean (stdev)	2.93 (1.74)

2.93% (exact two-sided Mann-Whitney U, $p \approx 0.69$), and both remain in complete separation below the static baseline's 22.2% ($p \approx 0.00794$ against the json-allowed arm too). This is a clean, decisive answer to the question Section IV-B could only partially resolve by conditioning on schema-evaluated documents: removing the sandbox's generation-overhead entirely, verified directly rather than inferred, does not close any measurable part of the RQ2 gap. The shortfall is a targeting problem, not a JSON-syntax-generation problem.

D. Follow-up: does richer feedback close the targeting gap?

Section IV-C established the gap is a targeting problem; the direct follow-up question is whether telling the proposer which categories of perturbation its divergence-causing documents last exhibited – missing field, wrong type, null override, unrecognized enum, boundary-magnitude id, extra key, deep nesting, computed automatically from already-collected data and fed back without naming specific schema fields – helps it target more effectively than the single numeric score alone. We ran a third n=5 arm (fresh seeds 200-204) under this richer prompt, otherwise identical to the main text's constrained sandbox.

Richer feedback did not help – it did measurably worse: a 0.95% mean divergence rate against the score-only prompt's 2.93%, a difference that holds under an exact two-sided Mann-Whitney U both over all generated documents ($p \approx 0.032$) and restricted to schema-evaluated documents only ($p \approx 0.032$, ruling out proposal-failure noise as the explanation). Two of the five category-feedback seeds found literally zero divergence in their only successfully-validated iteration, not merely zero because every iteration failed to produce data. We do not have a confirmed account of why more detailed feedback underperforms simpler feedback here; a plausible but untested reading is that a longer prompt enumerating seven named categories and their definitions competes for the proposer's attention with the actual code-generation task, or invites the model to write more elaborate, more bug-prone generator code attempting to explicitly target each named category, rather than the smaller, more robust generators the shorter prompt tends to produce. This result is small-sample (n=5) and reported descriptively for that reason, but it is a genuine data point against the intuitive assumption that more informative feedback is strictly better for this task.

TABLE V
RQ3: exact-integer round-trip on the id field, N=43,336 checks (all nesting depths, pooled across every RQ2 campaign).

Path	Accepted	Mismatched	Rate
Manual	10,070	0	0.0%
json_serializable	11,449	649	5.7%
freezed	11,449	649	5.7%
built_value	10,368	0	0.0%

E. RQ3: does numeric decoding preserve exact values?

Table V reports the ground-truth id comparison (Section III-D) computed as an analysis pass over schema-evaluated records already collected across every RQ2 campaign in this paper (static baseline, constrained-sandbox refined arm, and the Section IV-C ablation, pooled) – no separate generator, harness, or campaign was needed. The schema's recursive child field means an accepting path's canonical value mirrors the full decoded tree, not just its top-level fields, so every nesting depth already present in collected data is another independent check of the same kind; walking all of them (rather than only the top level, as an earlier pass over a smaller pool of campaigns did) raises N to 43,336 checks.

Manual parsing and built_value never silently mismatch: across ten thousand accepted decodes each, they either decode id exactly or reject the input outright. json_serializable and freezed are identical to each other (freezed delegates this field's decoding to json_serializable's generated code) and both show a 5.7% silent mismatch rate overall. Every observed mismatch saturates to exactly int64::MAX or int64::MIN – e.g. an input id of −9,223,372,036,854,775,810 decodes to −9,223,372,036,854,775,808 – consistent with the generated code calling (json['id'] as num).toInt() rather than json['id'] as int: an out-of-range JSON integer literal is first promoted to a double by dart:convert, and toInt() on an out-of-range double saturates silently rather than throwing.

Breaking json_serializable's rate out by nesting depth shows it is not uniform: 3.4% at depth 0 (n=8,198), rising to 7.7% at depth 1 (n=2,377), 20.7% at depth 2 (n=590), and 21.2% at depth 3 (n=240), before the sample thins out past depth 3 ($n \leq 23$ per level, too small to read

as more than suggestive). We do not have a confirmed mechanistic account of why deeper nesting correlates with a higher mismatch rate – a plausible but untested guess is that the generator's perturbation and recursion machinery are not independent, so documents that happen to nest deeply also happen to disproportionately carry boundary-magnitude id values – and report the pattern descriptively rather than as a tested effect of depth itself.

F. A targeted side study: does freezed ever diverge from json_serializable?

In every result reported above, freezed and json_serializable are identical, because freezed delegates every Record field's leaf-decoding to json_serializable's generated code (Section III-B) – the schema never exercises the one feature (native discriminated-union JSON dispatch) that is actually freezed's own. We ran a small, separate, deliberately qualitative side study – not folded into the RQ1-RQ3 statistical results, and not claiming their level of rigor – to check whether this identity holds even where it should not have to: a two-variant discriminated union, Payload.text(String) / Payload.number(num), discriminated by a "type" key, implemented once per path following each framework's own idiomatic pattern (manual: an if/else dispatch; json_serializable: a hand-written dispatch function plus @JsonSerializable() leaf classes, since json_serializable has no native polymorphism; freezed: @Freezed(unionKey: 'type'), its own built-in mechanism; built_value: a hand-written dispatch function plus a custom Serializer per leaf, since built_value has no native polymorphism either). Thirteen hand-picked documents (both valid variants, an unknown/missing/null/wrong-type/differently-cased discriminant, and wrong-type or missing/null leaf values) were run through all four paths.

The result: freezed finally diverges from json_serializable – but at the exception-type level, not accept/reject or value. Every invalid-discriminant case (unknown, missing, null, wrong-type, or differently-cased) is rejected by all four paths, but freezed raises CheckedFromJsonException (a public, structured, field-attributed error type from json_annotation's own checked-decoding support) while manual, json_serializable, and built_value all raise a generic ArgumentError – the first behavioral difference of any kind observed between B and C anywhere in this paper. For wrong-type or missing leaf-value cases (the discriminant itself is valid), all four paths converge back to the same pattern as the main schema: A/B/C raise the private _TypeError, D raises its own DeserializationError, and no B-vs-C difference appears. No accept/reject divergence and no value divergence (all four accepting but disagreeing on the decoded value) were observed in this small sample; the discriminant-based dispatch in all four paths is simple and deterministic enough that we would not expect the latter without a schema feature (e.g. ambiguous or overlapping variant matching) this minimal union does not have. We report this as a real, novel, practically-relevant qualitative finding – json_serializable's manual dispatch pattern for polymorphic JSON gives callers a less specific exception to catch than freezed's native union support does, for the exact failure mode (a malformed discriminant) polymorphic JSON is most likely to hit in practice – rather than as a quantitative result on the level of Sections IV-A–IV-D: formalizing it into an automated oracle, generator, and seeded campaign at scale is future work (Section V-E), not attempted here.

G. Two concrete, practitioner-actionable defects

Both of the following were found by the static generator alone, with no LLM involved, and are independent of the RQ1/RQ2 statistical comparisons above. (1) Silent integer saturation, RQ3 above: any Flutter application using json_serializable or freezed to decode an integer identifier (a snowflake ID, an external database key) that can exceed 64-bit range will silently receive 9223372036854775807 rather than an exception, for any such identifier, with no warning. (2) Silent default on a missing collection field: built_value decodes a document with the tags field entirely absent by silently defaulting it to an empty list, while manual parsing, json_serializable, and freezed all correctly reject the same input as malformed. This does not generalize to missing scalar fields – id, amount, and status are all correctly rejected as missing by every path including built_value – so the finding is precisely scoped to built_value's list-typed field construction, not general laxness. Both defects are reproducible with the released harness and require no fuzzing infrastructure to confirm by hand.

V. Discussion

A. Interpreting the RQ1/RQ2 contrast

The central empirical fact of this paper is not either number in Section IV alone, but that they point in opposite directions under an otherwise identical mechanism: the same refinement loop, the same proposer model, the same iteration budget, the same sandboxing discipline, raises acceptance rate on a like-for-like replication of the original study's objective (RQ1) and simultaneously underperforms a simple static generator on a differential divergence objective (RQ2). This is evidence that "LLM-guided refinement helps" is not a property of the pipeline in general – it is a property of the pipeline given a specific objective, and the objective matters more than we expected going in. Acceptance rate against a single decoder is a smooth, easy-to-improve-incrementally signal: almost any perturbation that makes a document more JSON-shaped moves the number in the right direction, and an LLM has presumably seen enormous amounts of well-formed JSON during training. Cross-implementation divergence is a much harder target: it requires a document

that is simultaneously well-formed enough to clear one implementation's more lenient path and malformed enough (in one specific, narrow way) to trip a stricter one, which is a needle-in-haystack search over a much smaller region of the input space than "more JSON-like" rewards.

We tested, rather than speculated about, the obvious alternative explanation: that RQ2's gap is mostly sandbox-driven generation overhead (the LLM cannot import a JSON library and must hand-build JSON text, which is failure-prone) rather than a genuine targeting shortfall. Directly permitting import json (Section III-J) eliminated the generation-overhead problem almost entirely – every ablation run reached at or near 100% schema_evaluated – yet the divergence rate did not move: 2.20% mean, statistically indistinguishable from the constrained sandbox's 2.93% ($p \approx 0.69$), both still an order of magnitude below the static baseline. The RQ2 gap is not generation overhead at all; it is entirely a targeting problem. This is a stronger and more precise conclusion than our first attempt at explaining it (restricting the comparison to schema-evaluated documents only, which could not fully separate the two effects).

Having localized the problem to targeting, the direct next question is whether more informative feedback about the targeting problem specifically closes it. It did not – and did measurably worse. Feeding back perturbation categories (Section III-K) produced a 0.95% mean divergence rate, significantly below the score-only prompt's 2.93% ($p \approx 0.032$, both overall and restricted to schema-evaluated documents, ruling out proposal-failure noise as the explanation). Taken together with the ablation, this suggests the limiting factor is not that the proposer lacks information about what tends to work – it is given exactly that information twice now, once implicitly (via the score) and once explicitly (via categories) – but a harder difficulty actually implementing an effective divergence-hunting generator in code, one that additional textual guidance does not straightforwardly fix and may compete with by adding more for the proposer to reason about at once. We report this as a genuine negative data point against the intuitive assumption that richer feedback is strictly better, not as a settled explanation of why: at n=5 we can establish the direction of the effect, not its mechanism.

B. The value of reporting a corrected prompt, not just a final one

Section III-I's prompt fix is itself worth discussing as a methodological point independent of its numeric effect. The unmodified prompt drove up an unrelated metric (per-path rejection counts) while completely failing at the intended one (divergence) across three full seeded runs – a proposer optimizing "move the numbers shown to me" when the numbers shown do not clearly identify which one is the actual objective. This mirrors, on a different metric, exactly the concern the original study's own discussion raised about acceptance-rate over-optimization crowding out boundary-probing behavior. We treat this as a reason to report negative or confounded intermediate results explicitly in future work on prompt-driven refinement objectives, rather than only the final, working design – the failure mode is informative on its own, and silently iterating past it would have understated how easy it is for an LLM-guided loop to optimize the wrong thing when the prompt does not make the objective unambiguous.

C. Threats to validity

Construct validity. RQ2's divergence oracle can only detect disagreement between the four specific paths instrumented; it says nothing about whether any individual path is "correct" relative to a specification, only that they disagree. RQ3's ground-truth oracle is narrower still, scoped to one field (id) chosen because it is the schema's only bare JSON-number token – a deliberate, evidenced choice (Section III) after an earlier draft of this design named a string-typed field that cannot exhibit numeric precision loss at all.

Internal validity. The n=5 RQ2 comparison used a corrected prompt that itself required an intermediate failed attempt to discover (Section III-I); we validated the fix on a held-out seed before committing the full five-seed budget to it specifically to guard against tuning the prompt to the same data used to report the result, but we did not attempt further prompt variants beyond the one fix, so we cannot rule out that a different, equally principled prompt design closes more of the RQ1/RQ2 gap than the one reported here.

Statistical validity. n=5 per arm is a small sample, chosen under a fixed, modest API budget and reported as exactly that throughout, following the original study's own precedent for its smaller-sample ablation. Complete rank separation at n=5 gives an exact $p \approx 0.00794$, which is the smallest attainable exact p-value this design can produce, not a measure of effect size independent of sample size; we report the underlying per-run rates in Tables I, II, III, and IV for exactly that reason, and a larger-n replication, budget permitting, is the direct way to strengthen this result (Section V-E).

External validity. All results are on JSON, on four specific, pinned-version Dart deserialization paths, with one LLM (gpt-4.1-mini) at one temperature setting. We make no claim that the magnitude of the RQ1, RQ2, or follow-up effects, or the RQ3 saturation pattern's exact threshold behavior, generalizes to other formats, other Dart packages or package versions, or other proposer models.

D. When this approach is, and is not, the right tool

The differential divergence oracle (RQ2) is only informative when multiple independent implementations of the same specification genuinely exist and are in real concurrent use, which is true of Dart JSON deserialization

specifically because the ecosystem has several competing, actively maintained codegen frameworks with no single dominant standard. A target with one canonical implementation and no independent alternative has no comparison partner for this oracle at all, and the numeric round-trip oracle (RQ3) or an exception-contract oracle (Section IV-G's private-exception classification) would be the applicable choice instead.

E. Future work

Several extensions follow directly from what this study measured but did not have budget to pursue further. First, since neither the score-only prompt nor the richer category-feedback prompt (Section III-K) closed the RQ2 gap – and the latter did measurably worse – the open question is no longer "what feedback helps" but whether the proposer's difficulty is better addressed structurally: for instance, letting it retain and mutate its previous iteration's generator source rather than rewriting from scratch each time (this pipeline's loop, reused unmodified from the original, always discards the prior proposal), or giving it a small library of worked examples of divergence-causing perturbations applied to an unrelated schema, rather than more description of the current one. Second, a larger-n replication of RQ1, RQ2, and both Section III-J/III-K follow-ups, budget permitting, would tighten the exact p-values reported here past what n=5 can produce, the same way the original study's own n=15 result tightened its earlier smaller-sample finding. Third, the qualitative union side study (Section IV-F) found a real exception-type difference between freezed and json_serializable on thirteen hand-picked documents but was not run through this paper's automated oracle, generator, or seeded-campaign machinery at all; building a Payload-shaped generator and harness endpoint and re-running the RQ2-style baseline-vs-refined comparison on it would establish whether this exception-type difference (or a genuine value divergence, not observed in the small hand-picked sample) appears at the rate and statistical rigor the main Record schema's findings do, rather than resting on thirteen cases.

VI. Conclusion

We took up a generalization axis a prior study named explicitly as future work – retargeting an LLM-guided, coverage-free grammar-fuzzing pipeline at a memory-safe language, where its original sanitizer-based crash oracle has no meaning – and designed two replacements that need no crash at all: cross-implementation differential divergence across four widely-used Dart JSON deserialization paths, and a ground-truth numeric round-trip check. Reusing the original refinement loop with no change to its mechanism, we found that it reproduces the original acceptance-rate result almost exactly on this new target (50.0% to a 92.0% mean, complete rank separation, exact $p \approx 0.00794$), confirming the mechanism itself generalizes across runtime and language. The same mechanism, same model, same iteration budget, did not transfer to our differential objective: even after a confirmed, non-arbitrary prompt correction that isolated divergence as the explicit score, refinement found divergence in a 2.9% mean of generated documents against a static, schema-aware generator's 22.2% (complete rank separation in the opposite direction, exact $p \approx 0.00794$). We take this contrast – not either result alone – to be the paper's central methodological finding: whether LLM-guided refinement helps depends on the objective it is asked to optimize, holding everything else about the pipeline fixed. A follow-up ablation tested, rather than assumed, the obvious alternative explanation for the RQ2 gap: lifting the sandbox's restriction against importing a JSON library eliminated the LLM's generation-quality overhead almost entirely – every ablation run reached at or near 100% syntactically valid output – yet left the divergence rate statistically unchanged (2.2% mean, $p \approx 0.69$ against the constrained sandbox, still $p \approx 0.00794$ against the baseline). The RQ2 shortfall is a targeting problem, not a generation-quality one, decisively. A second follow-up then tested the natural next hypothesis – that feeding the proposer perturbation categories rather than a bare score would help it target more effectively – and found the opposite: a measurably worse divergence rate (0.95% mean, $p \approx 0.032$ against the score-only prompt, holding under a check restricted to schema-evaluated documents), a genuine negative data point against the assumption that more informative feedback is strictly better for this task.

Two concrete, practitioner-actionable defects fell out of the static generator's own campaigns, independent of any LLM: built_value silently defaults a missing list-typed required field to empty rather than rejecting the input, and json_serializable and freezed both silently saturate an out-of-range integer field to int64::MIN/MAX with no exception at all, confirmed at a 5.7% mismatch rate across 43,336 sampled checks by re-analyzing data already collected for RQ2 rather than running a fourth, redundant pipeline. Either defect is directly actionable for a Flutter team choosing between these frameworks or handling externally-sourced numeric identifiers, independent of anything this paper argues about LLM-guided refinement.

We release the full pipeline – the four-path Dart harness, both generators, the refinement loop and both its corrected prompt and its category-feedback variant, and every campaign artifact underlying Tables I, II, III, IV, and V – so that the RQ1/RQ2 contrast, the two follow-ups on the RQ2 gap, the RQ3 saturation pattern, and the two concrete defects can all be checked, extended to a larger sample, or falsified directly rather than taken on faith.

References

[1] Z. M. M. Ibrahim, "Agentic grammar fuzzing: LLM-guided grammar refinement for parser testing," https://github.com/ziyadmansy/agentic-grammar-fuzzing, 2026, version 1.1.0.

[2] M. Zalewski, "American fuzzy lop," https://lcamtuf.coredump.cx/afl/, 2014, accessed 2026.
[3] A. Fioraldi, D. C. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association, 2020.
[4] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," IEEE Transactions on Software Engineering, vol. 47, no. 11, pp. 2312–2331, 2021.
[5] P. Purdom, "A sentence generator for testing parsers," BIT Numerical Mathematics, vol. 12, no. 3, pp. 366–375, 1972.
[6] J. Wang, B. Chen, L. Wei, and Y. Liu, "Superion: Grammar-aware greybox fuzzing," in Proceedings of the 41st International Conference on Software Engineering (ICSE), 2019, pp. 724–735.
[7] C. Aschermann, T. Frassetto, T. Holz, P. Jauernig, A.-R. Sadeghi, and D. Teuchert, "NAUTILUS: Fishing for deep bugs with grammars," in Proceedings of the 26th Network and Distributed System Security Symposium (NDSS), 2019.
[8] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP), 2000, pp. 268–279.
[9] D. R. MacIver, Z. Hatfield-Dodds, and Many other contributors, "Hypothesis: A new approach to property-based testing," Journal of Open Source Software, vol. 4, no. 43, p. 1891, 2019.
[10] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2023, pp. 423–435.
[11] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, and L. Zhang, "Fuzz4All: Universal fuzzing with large language models," in Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2024.
[12] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CodaMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in Proceedings of the 45th International Conference on Software Engineering (ICSE), 2023.
[13] W. M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, pp. 100–107, 1998.
[14] "The Dart programming language," https://dart.dev, 2026, SDK version 3.13.2, accessed 2026.
[15] "json_serializable: Automatically generate code for converting to and from JSON," https://pub.dev/packages/json_serializable, 2026, version 6.14.1.
[16] R. Rousselet, "freezed: Code generation for immutable classes," https://pub.dev/packages/freezed, 2026, version 4.0.1.
[17] "built_value: Value types with builders, Dart classes as enums, and serialization," https://pub.dev/packages/built_value, 2026, version 8.13.0.
[18] "ANTLR grammars-v4: JSON grammar," https://github.com/antlr/grammars-v4/blob/master/json/JSON.g4, 2024.
[19] T. Parr, The Definitive ANTLR 4 Reference, 2nd ed. Pragmatic Bookshelf, 2013.